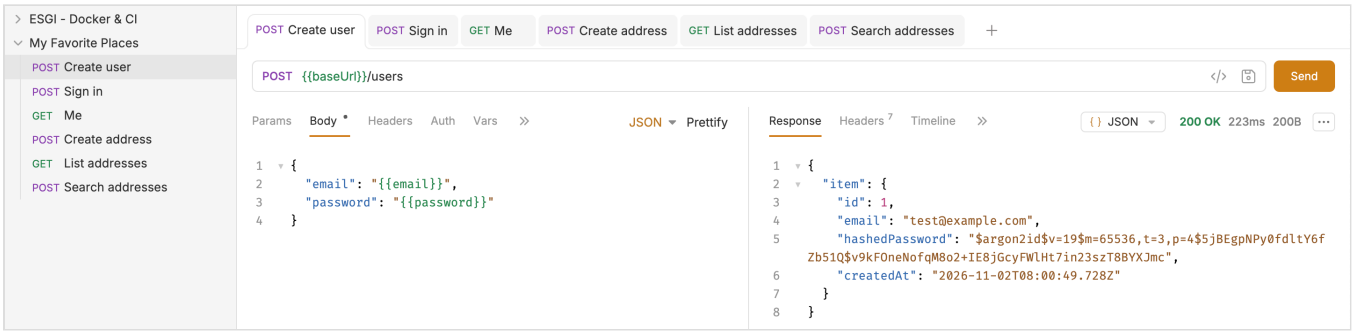
Clusterisation de conteneurs : My Favorite Places

1 - Introduction

TD 1 : dockerisation du projet

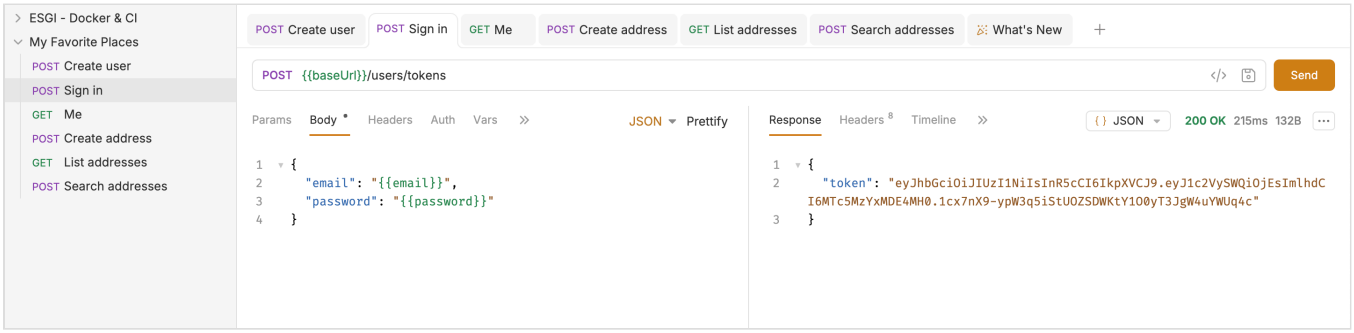
1. Tester le serveur avec `yarn dev` et faire une requête avec Bruno

```
(base) ➜  server git:(main) ✗ yarn dev
yarn run v1.22.22
$ nodemon src/index.ts
[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: ts,json
[nodemon] starting `ts-node src/index.ts`
⌛ starting server!
query: SELECT version()
query: SELECT * FROM current_schema()
query: START TRANSACTION
query: SELECT * FROM current_schema()
query: SELECT * FROM current_database()
query: SELECT "table_schema", "table_name", obj_description(('"' || "table_schema" || '"."' || "table_name" || '"')::regclass, 'pg_class') AS table_comment FROM "information_schem
a"."tables" WHERE ("table_schema" = 'public' AND "table_name" = 'address') OR ("table_schema" = 'public' AND "table_name" = 'user')
query: SELECT * FROM "information_schema"."tables" WHERE "table_schema" = 'public' AND "table_name" = 'typeorm_metadata'
query: CREATE TABLE "address" ("id" SERIAL NOT NULL, "name" character varying NOT NULL, "description" character varying, "lng" double precision NOT NULL, "lat" double precision NO
T NULL, "createdAt" TIMESTAMP NOT NULL DEFAULT now(), "userId" integer, CONSTRAINT "PK_d92de1f82754668b5f5f5dd4fd5" PRIMARY KEY ("id"))
query: CREATE TABLE "user" ("id" SERIAL NOT NULL, "email" character varying NOT NULL, "hashedPassword" character varying NOT NULL, "createdAt" TIMESTAMP NOT NULL DEFAULT now(), CO
NSTRAINT "PK_cace4a159ff9f2512dd42373760" PRIMARY KEY ("id"))
query: ALTER TABLE "address" ADD CONSTRAINT "FK_d25f1ea79e282cc8a42bd616aa3" FOREIGN KEY ("userId") REFERENCES "user"("id") ON DELETE NO ACTION ON UPDATE NO ACTION
query: COMMIT
✅ connected to PgSQL db
🚀 server started on port 3000
```



2. Dockeriser le serveur et retester avec Bruno

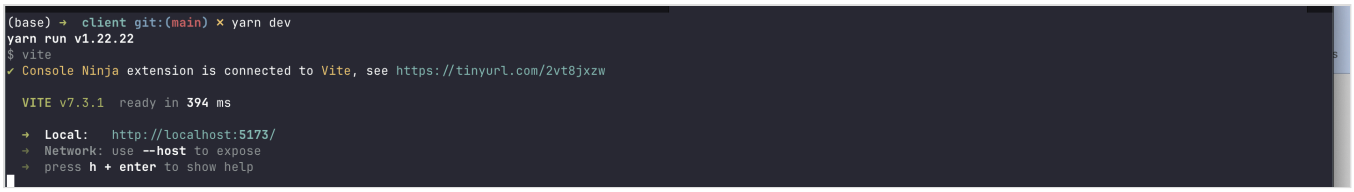
```
(base) ➜  server git:(main) ✗ docker ps
CONTAINER ID   IMAGE              COMMAND                  CREATED          STATUS          PORTS                    NAMES
a21dd3a84976   mfp-server         "docker-entrypoint.s…"   5 seconds ago    Up 5 seconds    0.0.0.0:3000->3000/tcp   mfp-server-tmp
366eddbda0a1   postgres:17-alpine "docker-entrypoint.s…"   3 minutes ago    Up 3 minutes    0.0.0.0:5432->5432/tcp   mfp-pg-tmp
```

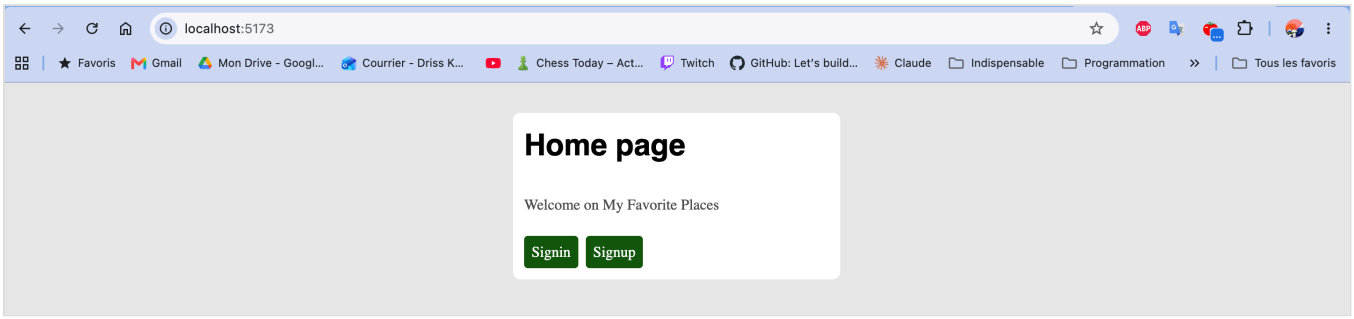


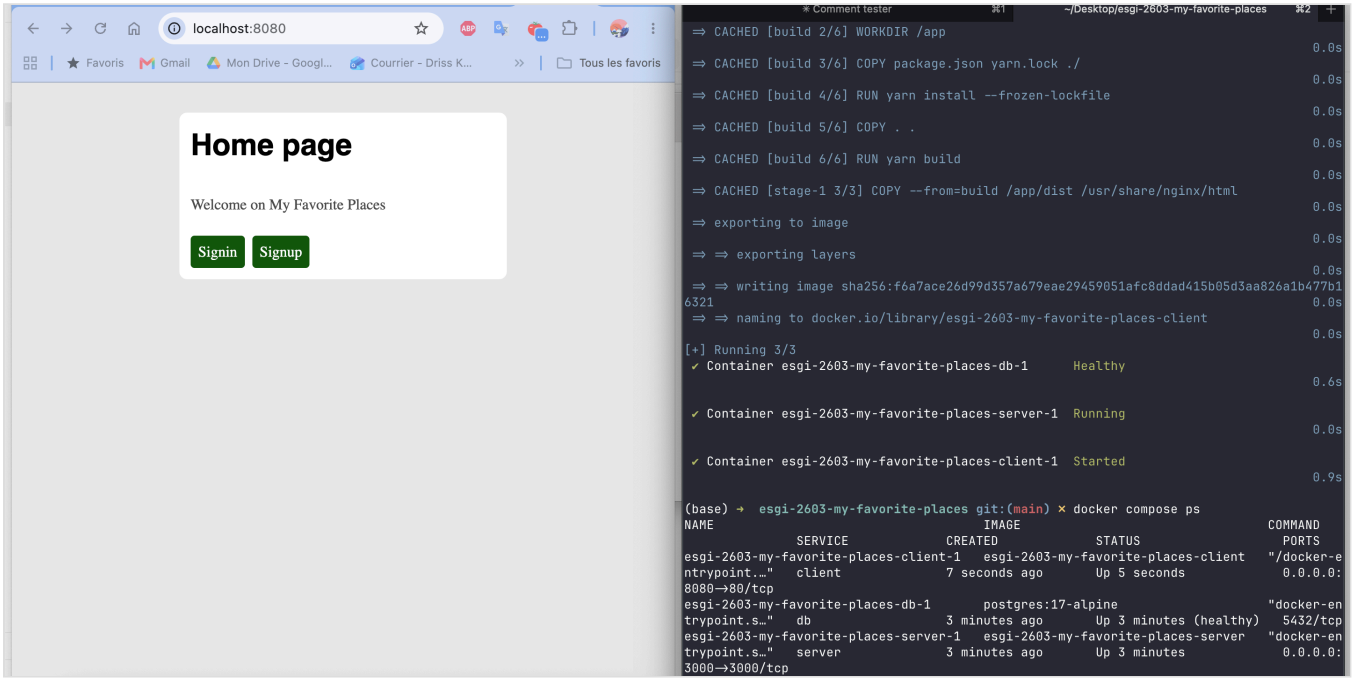
3. `compose.yml` qui lance le serveur et une BDD PostgreSQL

```
[+] Running 3/3
 ✔ Network esgi-2603-my-favorite-places_default     Created                                                                0.0s
 ✔ Container esgi-2603-my-favorite-places-db-1      Healthy                                                                6.0s
 ✔ Container esgi-2603-my-favorite-places-server-1  Started                                                                6.2s
(base) ➜  esgi-2603-my-favorite-places git:(main) ✗ docker compose ps
NAME                                    IMAGE                                  COMMAND                  SERVICE   CREATED          STATUS                    PORTS
esgi-2603-my-favorite-places-db-1       postgres:17-alpine                     "docker-entrypoint.s…"   db        8 seconds ago    Up 7 seconds (healthy)    5432/tcp
esgi-2603-my-favorite-places-server-1   esgi-2603-my-favorite-places-server    "docker-entrypoint.s…"   server    8 seconds ago    Up 2 seconds              0.0.0.0:3
000->3000/tcp
```

4. Tester le client React, le dockeriser et l'ajouter au `compose.yml`







5. Comment communiquent la BDD et le serveur ? Le serveur et le front ?

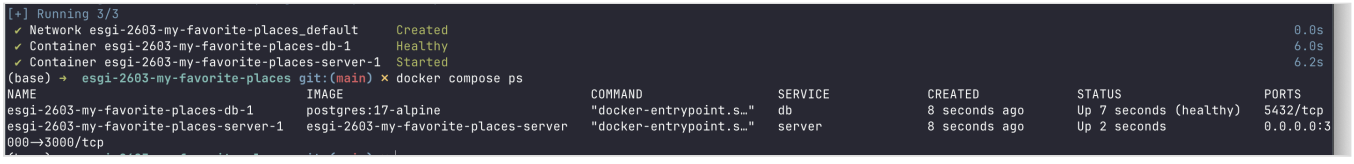
BDD et serveur : par le réseau Docker de Compose, le serveur joint la base avec le nom du service, `db:5432`.

Serveur et front : le navigateur appelle `/api` sur le front, et Nginx transmet la requête à `server:3000`.

TD 2

1. Démarrer le serveur une fois la BDD prête

`healthcheck` sur `db` et `depends_on: condition: service_healthy` sur `server` : `db` passe `Healthy`, puis `server` démarre.



2. Redémarrage automatique, sauf arrêt manuel

`restart: unless-stopped` sur les 3 services de `compose.yml`.

3. Build stages et image du serveur allégée d'environ 100 Mo

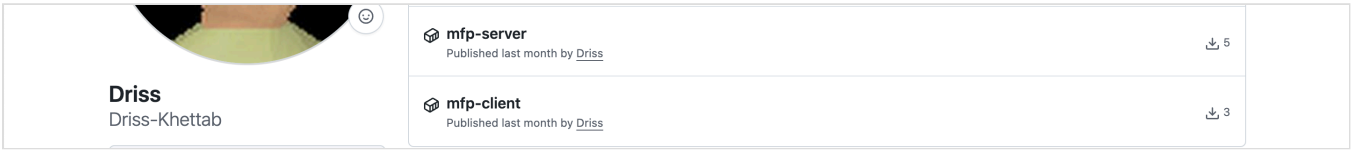
```
REPOSITORY   TAG         IMAGE ID       CREATED          SIZE                    mfp-server   naive       9cd16d316c7c   18 seconds ago   314MB
mfp-server   latest      24c423cb7472   3 weeks ago      200MB                   mfp-server   optimized   24c423cb7472   3 weeks ago      200MB
(base) ➜  esgi-2603-my-favorite-places git:(main) ×
```

2 - Docker Swarm

TD 1 : DinD et DooD

DinD fait tourner un daemon Docker complet dans un conteneur, DooD réutilise celui de l'hôte. Pour tester Swarm, chaque nœud est un conteneur DinD : chacun a son propre daemon, comme une machine séparée.

Prérequis : images front et back sur un registry



TD 2 : création du cluster

1. `compose.yml` avec 1 manager et 3 nodes DinD, `docker ps` dans un conteneur

```
docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED       STATUS       PORTS                    NAMES
aae95a03e987   docker:dind    "dockerd-entrypoint.…"   2 hours ago   Up 2 hours   2375-2376/tcp            swarm-node-1
807b7be84c6a   docker:dind    "dockerd-entrypoint.…"   2 hours ago   Up 2 hours   2375-2376/tcp            swarm-node-2
645482f411fd   docker:dind    "dockerd-entrypoint.…"   2 hours ago   Up 2 hours   2375-2376/tcp            swarm-node-3
acbb0c08daca   docker:dind    "dockerd-entrypoint.…"   2 hours ago   Up 2 hours   2375-2376/tcp            swarm-manager-1
/ # docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
/ #
```

2. Initialiser le cluster sur le manager

3. Faire rejoindre les nodes

4. Lister les nœuds depuis le manager

```
(base) ➜  swarm git:(main) × docker exec swarm-node-1 docker swarm join --token SWMTKN-1-2xellce1nj1f0bj4q4pr9ue9re3w9ro5agbuxr9j8zi0dzfwyk-cxw5eitorxxl9rg3gv4nmq41v manager:2377
This node joined a swarm as a worker.
(base) ➜  swarm git:(main) × docker exec swarm-node-2 docker swarm join --token SWMTKN-1-2xellce1nj1f0bj4q4pr9ue9re3w9ro5agbuxr9j8zi0dzfwyk-cxw5eitorxxl9rg3gv4nmq41v manager:2377
This node joined a swarm as a worker.
(base) ➜  swarm git:(main) × docker exec swarm-node-3 docker swarm join --token SWMTKN-1-2xellce1nj1f0bj4q4pr9ue9re3w9ro5agbuxr9j8zi0dzfwyk-cxw5eitorxxl9rg3gv4nmq41v manager:237
^C
(base) ➜  swarm git:(main) × docker exec swarm-node-3 docker swarm join --token SWMTKN-1-2xellce1nj1f0bj4q4pr9ue9re3w9ro5agbuxr9j8zi0dzfwyk-cxw5eitorxxl9rg3gv4nmq41v manager:2377
This node joined a swarm as a worker.
(base) ➜  swarm git:(main) × docker exec swarm-manager-1 docker node ls
ID                            HOSTNAME       STATUS    AVAILABILITY   MANAGER STATUS   ENGINE VERSION
9h3f11axmlchtmm0i71vp9i1g     8907e3dd6a48   Ready     Active                          29.8.2
yz12wfvmdqo3vgv61iw1wwjla     a7942e19ec88   Ready     Active                          29.8.2
jnf1tex2bfpixe7c9vii3dxtb     b86bbbd9caee   Ready     Active                          29.8.2
uqrtrwqcj69w44wzz8ndefz1z *   manager        Ready     Active         Leader           29.8.2
(base) ➜  swarm git:(main) ×
```

TD 3 : tests du cluster

1. Stack `hello-world.compose.yml` avec 2 réplicas

2. Copier la stack dans `/home/manager` sur le manager

```
(base) ➜  swarm git:(main) × docker exec swarm-manager-1 mkdir -p /home/manager
(base) ➜  swarm git:(main) × docker cp hello-world.compose.yml swarm-manager-1:/home/manager/
(base) ➜  swarm git:(main) × docker exec swarm-manager-1 ls -l /home/manager
total 4
-rw-r--r--    1 501      dialout         72 Nov  2 07:35 hello-world.compose.yml
(base) ➜  swarm git:(main) × docker exec swarm-manager-1 docker stack deploy -c /home/manager/hello-world.compose.yml hello
Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating network hello_default
Creating service hello_hello
(base) ➜  swarm git:(main) ×
```

3. `docker service ls` et `docker ps` sur chaque nœud : qu'en déduisez-vous ?

```
(base) ➜  swarm git:(main) ✗ docker exec swarm-manager-1 docker service ls
ID             NAME          MODE         REPLICAS   IMAGE               PORTS
ilv8tk9p1960   hello_hello   replicated   0/2        hello-world:latest
(base) ➜  swarm git:(main) ✗ docker exec swarm-manager-1 docker ps -a
CONTAINER ID   IMAGE                COMMAND     CREATED          STATUS                      PORTS     NAMES
5a94fc3e3955   hello-world:latest   "/hello"    3 seconds ago    Created                               hello_hello.2.ymwnw229ofo1xuvkyt9j3z9tr
d2f5913a968b   hello-world:latest   "/hello"    10 seconds ago   Exited (0) 4 seconds ago              hello_hello.1.1piu5bwb7daadyaugpnix5rmm
6890b0cc832b   hello-world:latest   "/hello"    20 seconds ago   Exited (0) 15 seconds ago             hello_hello.1.uj0qyn3izah597ud2v2le3or5
a26eccc3db8c   hello-world:latest   "/hello"    26 seconds ago   Exited (0) 21 seconds ago             hello_hello.1.irndcxoy39x38ofncq7bfdto0
(base) ➜  swarm git:(main) ✗ docker exec swarm-node-1 docker ps -a
docker exec swarm-node-2 docker ps -a
docker exec swarm-node-3 docker ps -a
CONTAINER ID   IMAGE                COMMAND     CREATED          STATUS                      PORTS     NAMES
32266fcb4db4   hello-world:latest   "/hello"    15 seconds ago   Exited (0) 9 seconds ago              hello_hello.2.7cumparvikcjfpu3hvu9625rn
7981c8598144   hello-world:latest   "/hello"    26 seconds ago   Exited (0) 21 seconds ago             hello_hello.2.l8gqw3uibs92iv81frl8okp2k
CONTAINER ID   IMAGE                COMMAND     CREATED          STATUS                                PORTS     NAMES
fc97bbf7f83e   hello-world:latest   "/hello"    6 seconds ago    Exited (0) Less than a second ago               hello_hello.1.kdka0tnzx5ias8ijutlui5vbp
870bba535a74   hello-world:latest   "/hello"    11 seconds ago   Exited (0) 5 seconds ago                        hello_hello.1.ko9b21axn2nt6fx4hp475n54o
f311521c1e57   hello-world:latest   "/hello"    22 seconds ago   Exited (0) 16 seconds ago                       hello_hello.1.czn16lqb998xf640cn14vzz9u
CONTAINER ID   IMAGE                COMMAND     CREATED          STATUS                      PORTS     NAMES
ebd859505756   hello-world:latest   "/hello"    21 seconds ago   Exited (0) 15 seconds ago             hello_hello.2.lla0qymvn4xya2ttwphqklhq5
(base) ➜  swarm git:(main) ✗
```

Swarm répartit les conteneurs sur les nœuds. Chaque nœud ne voit que les siens, et seul le manager a la vue d'ensemble. `hello-world` s'arrête aussitôt, donc Swarm le relance en boucle.

4. Faire varier `deploy` : manager seulement, puis global

Fichiers `swarm/hello-world.manager.compose.yml` et `swarm/hello-world.global.compose.yml`.

4 - Traefik et Portainer

Exercice 1 : déployer Traefik

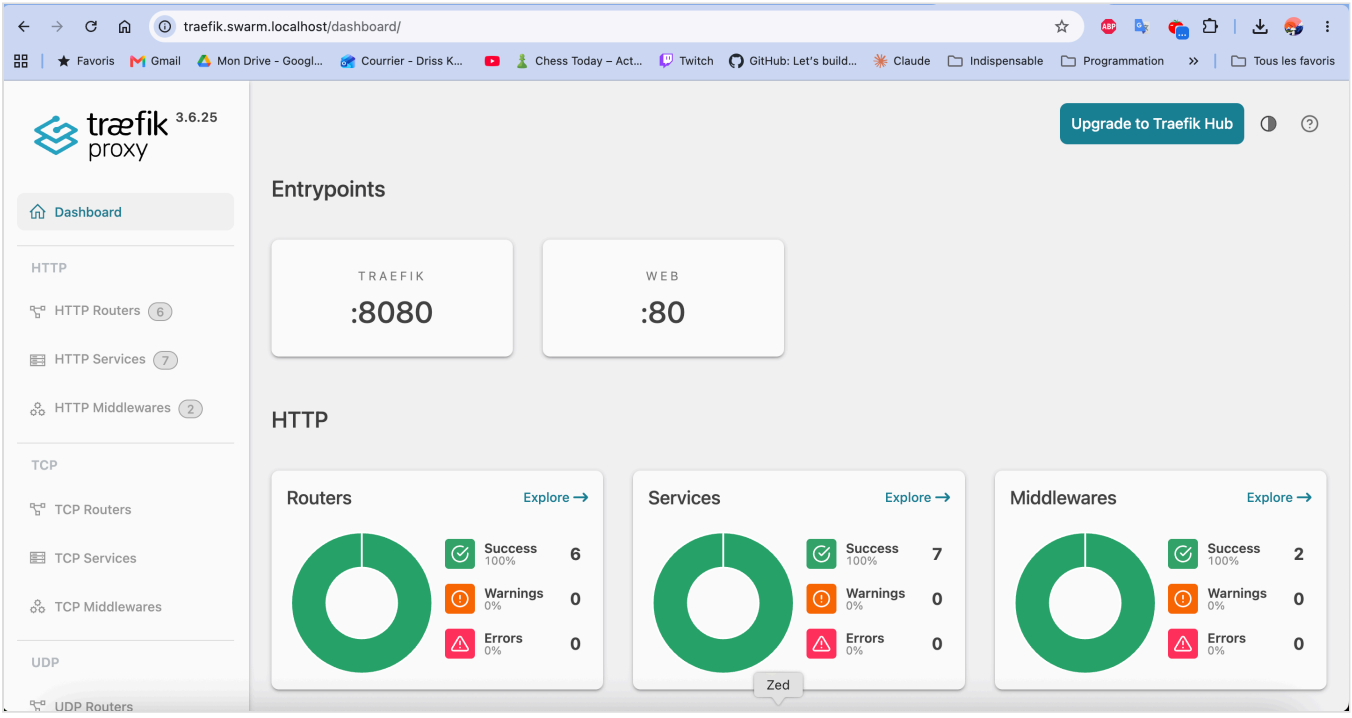
1. Relancer le cluster Swarm

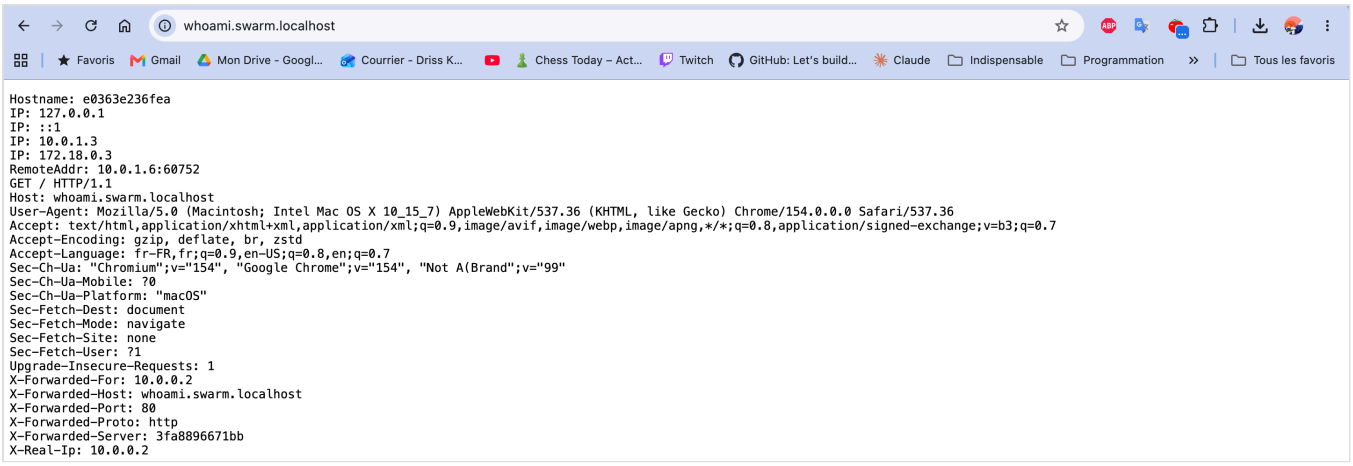
2. Déployer la stack sur le manager, avec le réseau `web` en overlay

```
(base) ➜  example-voting-app git:(main) docker exec swarm-manager-1 docker stack services traefik
ID             NAME              MODE         REPLICAS   IMAGE                PORTS
xv7niwjzl0hl   traefik_traefik   replicated   1/1        traefik:v3.6         *:80→80/tcp
khtutjpw10ef   traefik_whoami    replicated   1/1        traefik/whoami:latest
```

3. Fichier hosts : `traefik.swarm.localhost` et `whoami.swarm.localhost`

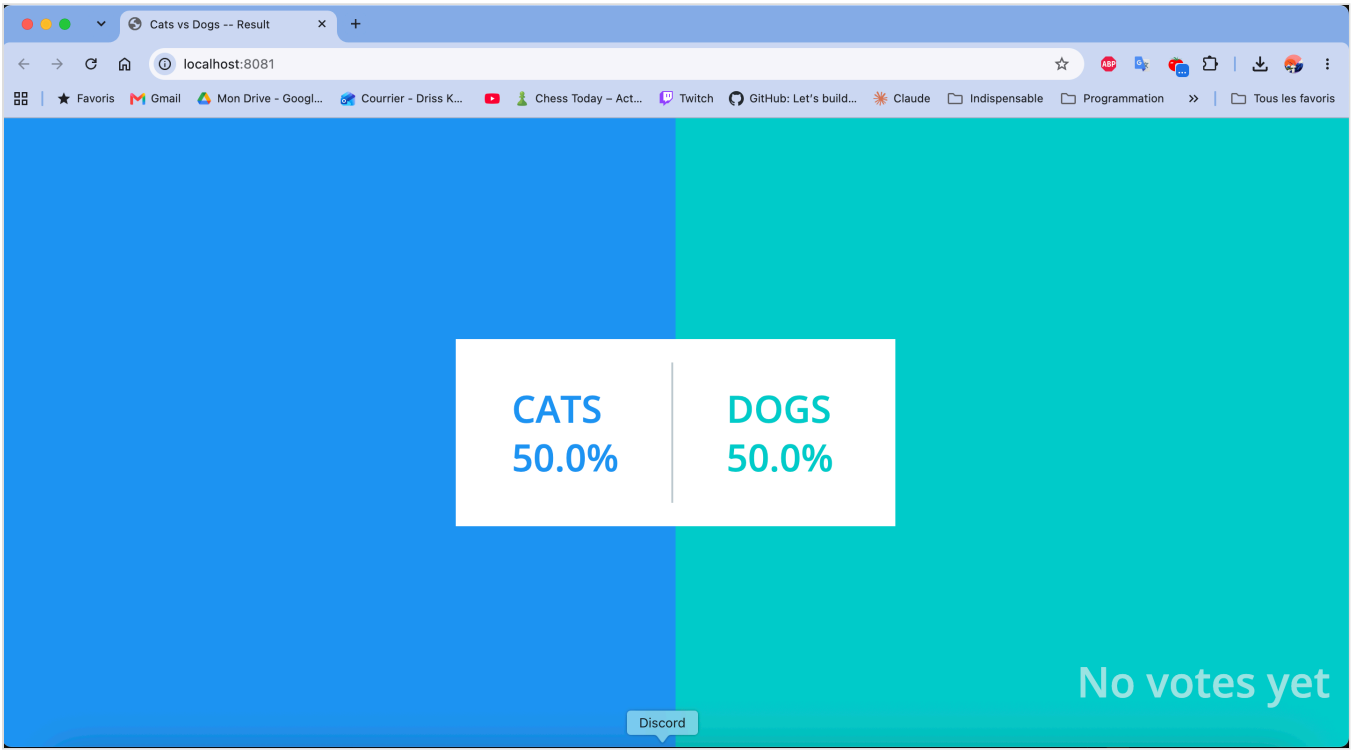
```
(base) ➜  swarm git:(main) ✗ grep swarm /etc/hosts
127.0.0.1 traefik.swarm.localhost whoami.swarm.localhost vote.swarm.localhost result.swarm.localhost
(base) ➜  swarm git:(main) ✗
```





Exercice 2 : déployer une autre application

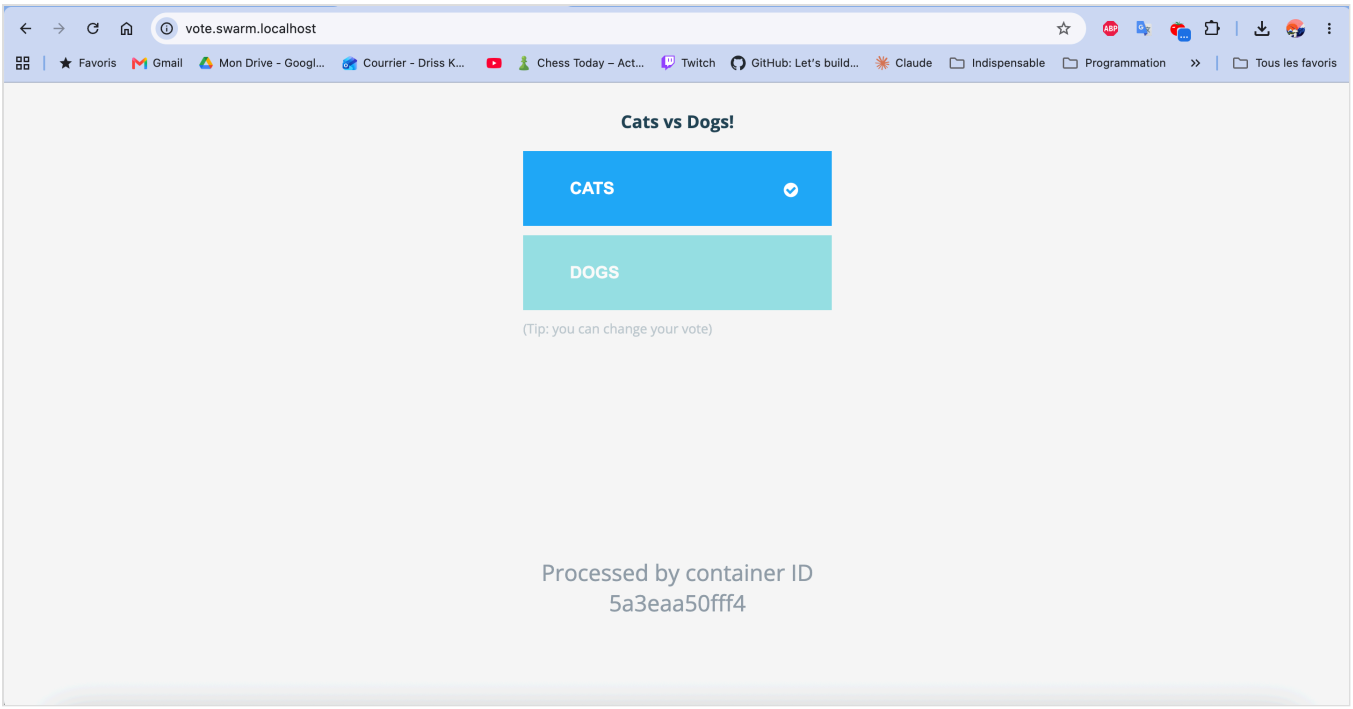
1. Tester l'application avec Docker Compose

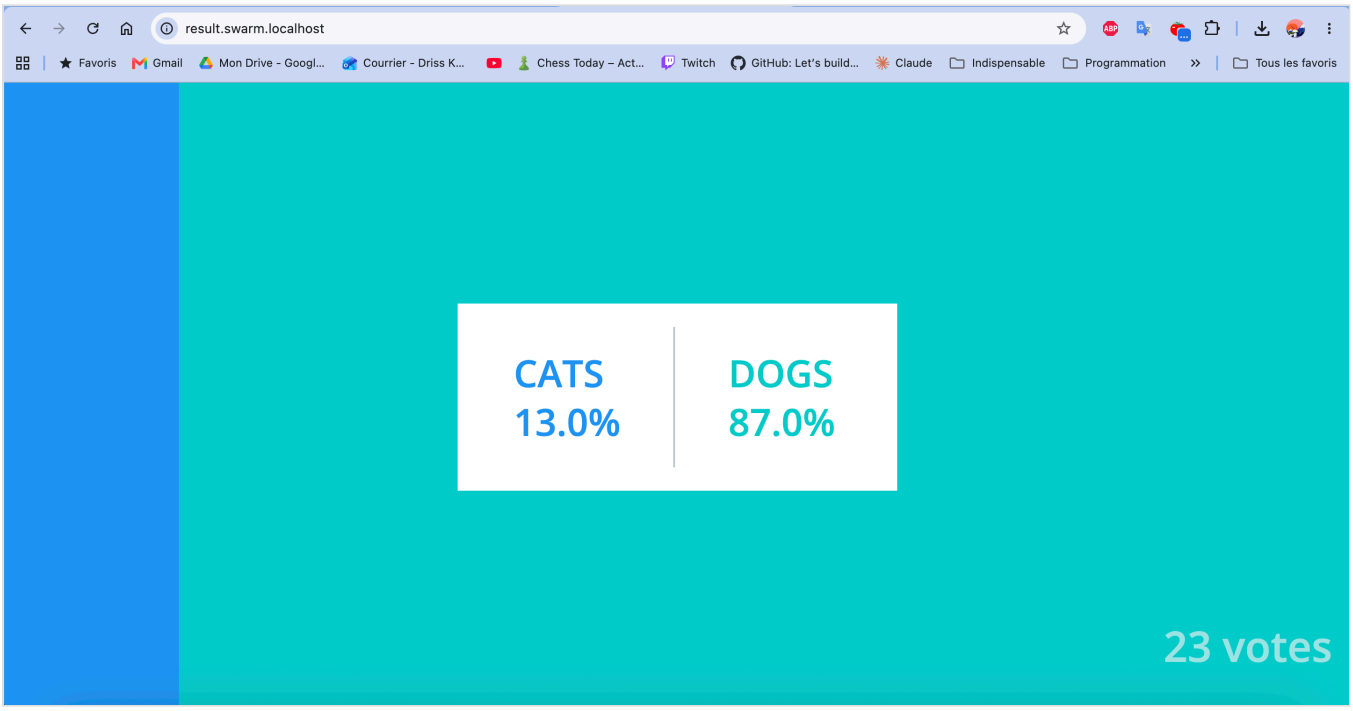


2. La déployer sur Swarm

```
(base) →  swarm git:(main) ✗ docker exec swarm-manager-1 docker stack ps vote --filter desired-state=running
ID             NAME            IMAGE                                        NODE          DESIRED STATE   CURRENT STATE           ERROR     PORTS
ip2n699l2y75   vote_db.1       postgres:15-alpine                           b86bbbd9caee   Running         Running 53 seconds ago
h7awzaatcq9l   vote_result.1   dockersamples/examplevotingapp_result:latest   manager       Running         Running 52 seconds ago
tlf7xvay5tqt   vote_vote.1     dockersamples/examplevotingapp_vote:latest     b86bbbd9caee   Running         Running 57 seconds ago
```

3. Vote et résultats sur 2 URLs





Exercice 3 : déployer Portainer

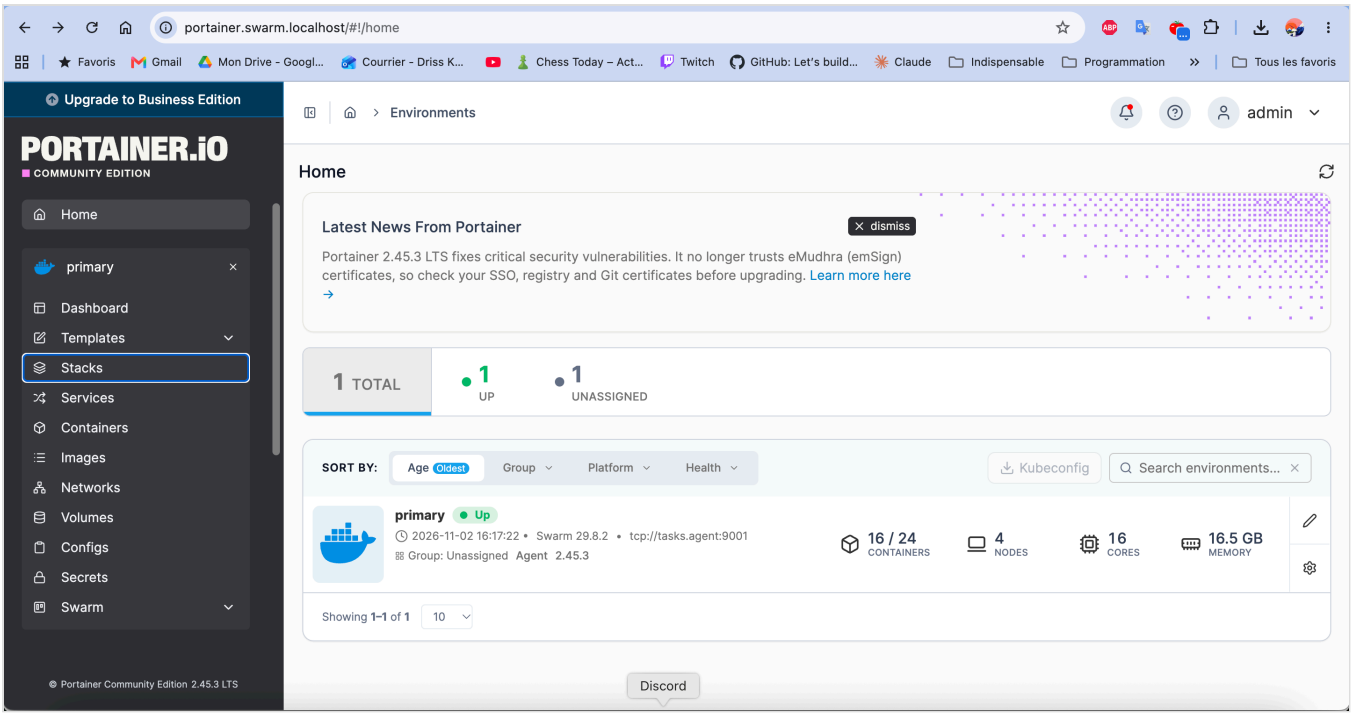
1. Déployer Portainer sur Swarm

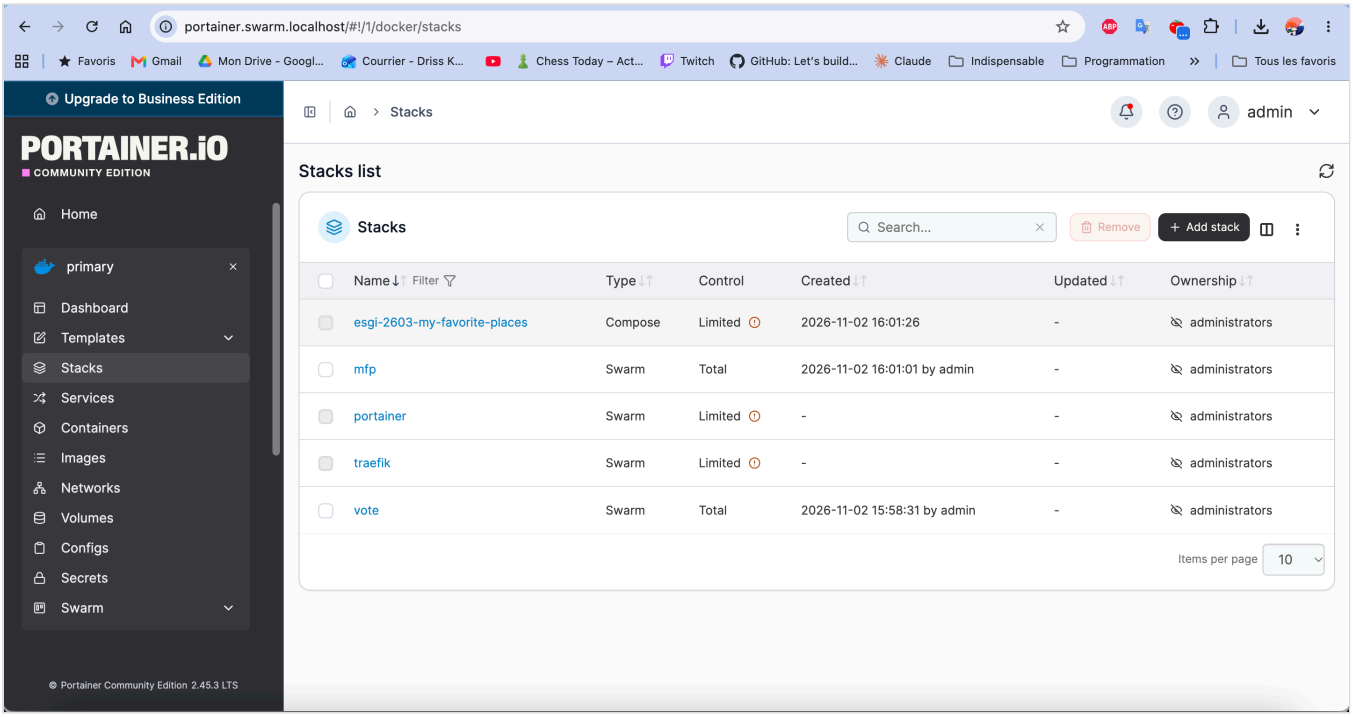
```
(base) ➜  esgi-2603-my-favorite-places git:(main) ✗ docker exec swarm-manager-1 docker stack services portainer
ID             NAME                 MODE         REPLICAS   IMAGE                          PORTS
prv488cyf04q   portainer_agent      global       4/4        portainer/agent:lts
jbzexg9mprou   portainer_portainer   replicated   1/1        portainer/portainer-ce:lts
```

2. Publier Portainer sur une URL avec des labels

3. Mettre à jour le fichier hosts et retrouver ses stacks

```
(base) ➜  esgi-2603-my-favorite-places git:(main) ✗ grep swarm /etc/hosts
127.0.0.1 traefik.swarm.localhost whoami.swarm.localhost vote.swarm.localhost result.swarm.localhost
127.0.0.1 portainer.swarm.localhost mfp-api.swarm.localhost
```



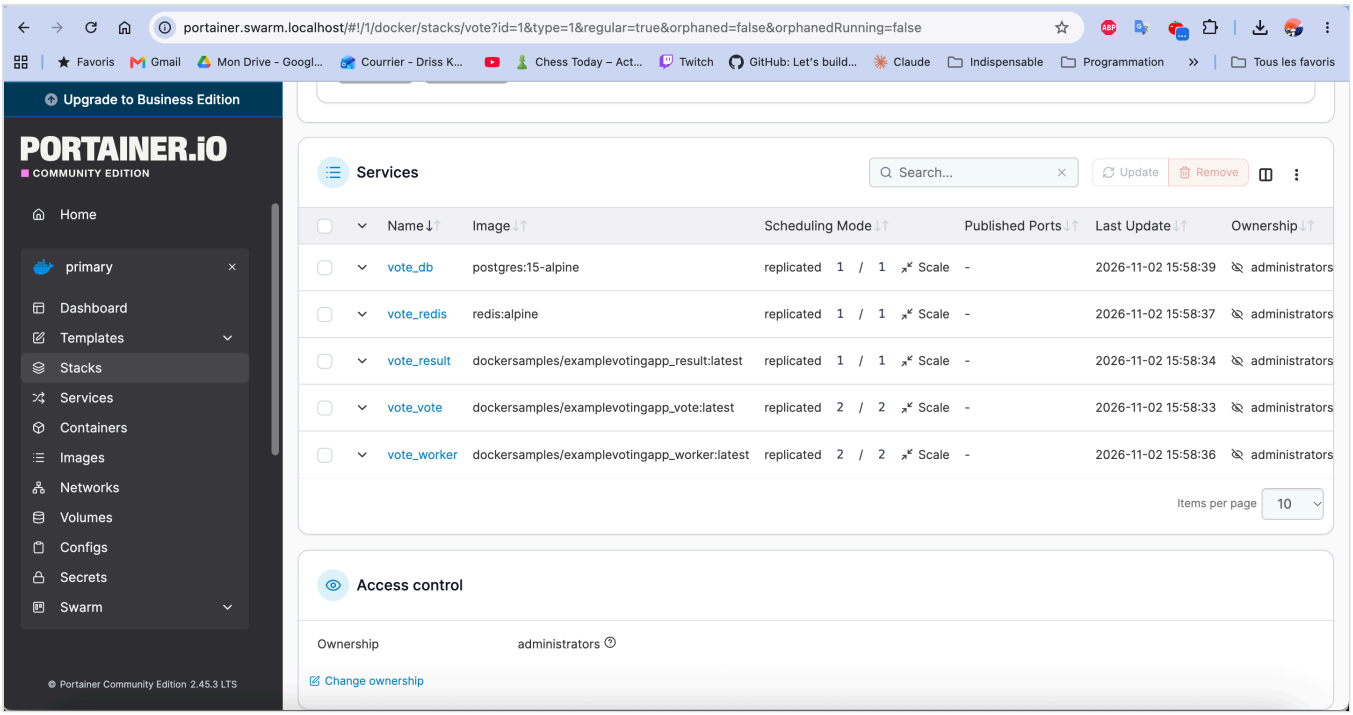


Exercice 4 : redéployer l'app de vote

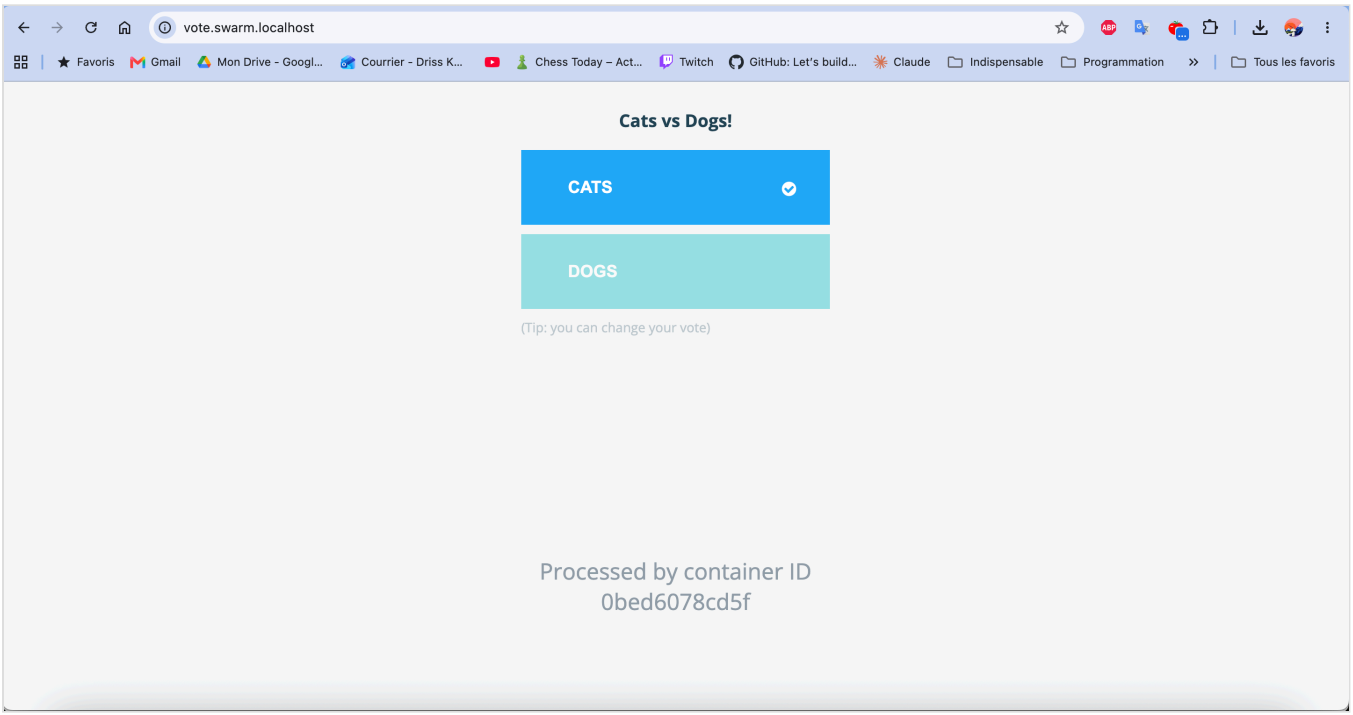
1. Supprimer la stack de vote

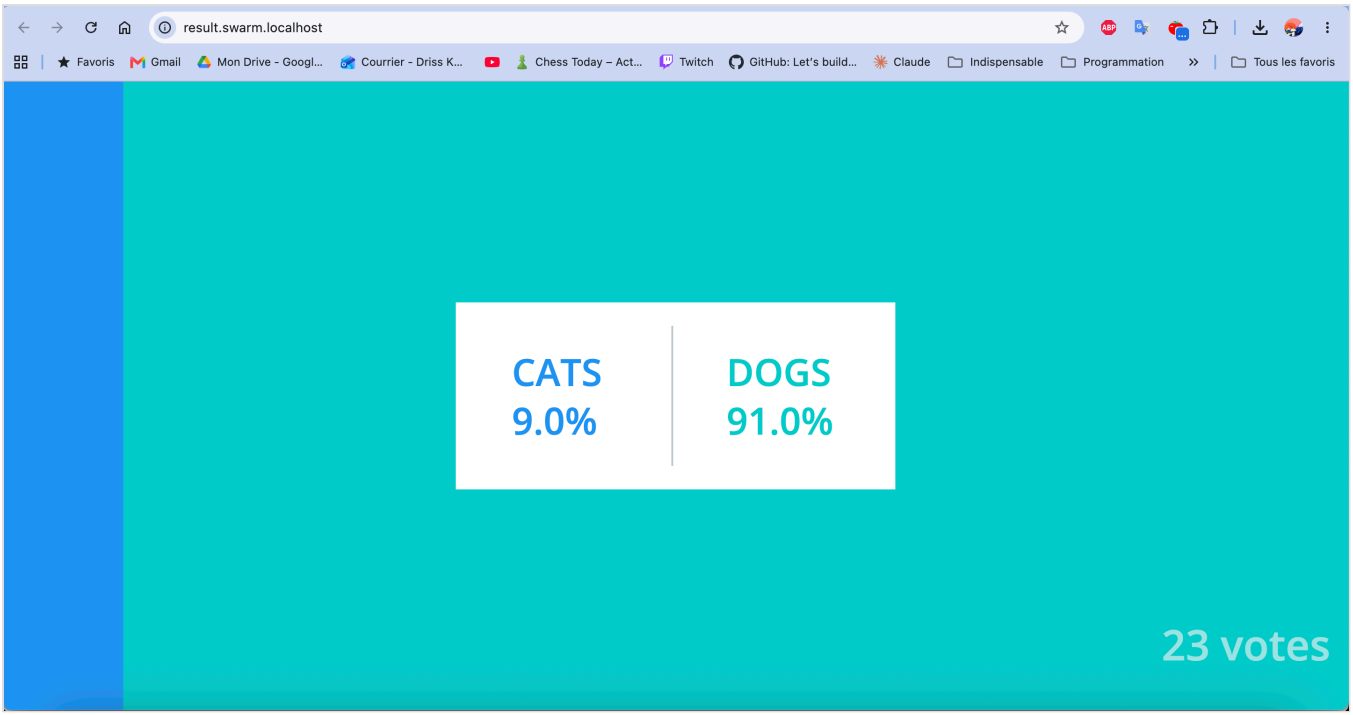
```
docker exec swarm-manager-1 docker stack rm vote
```

2. La republier depuis Portainer

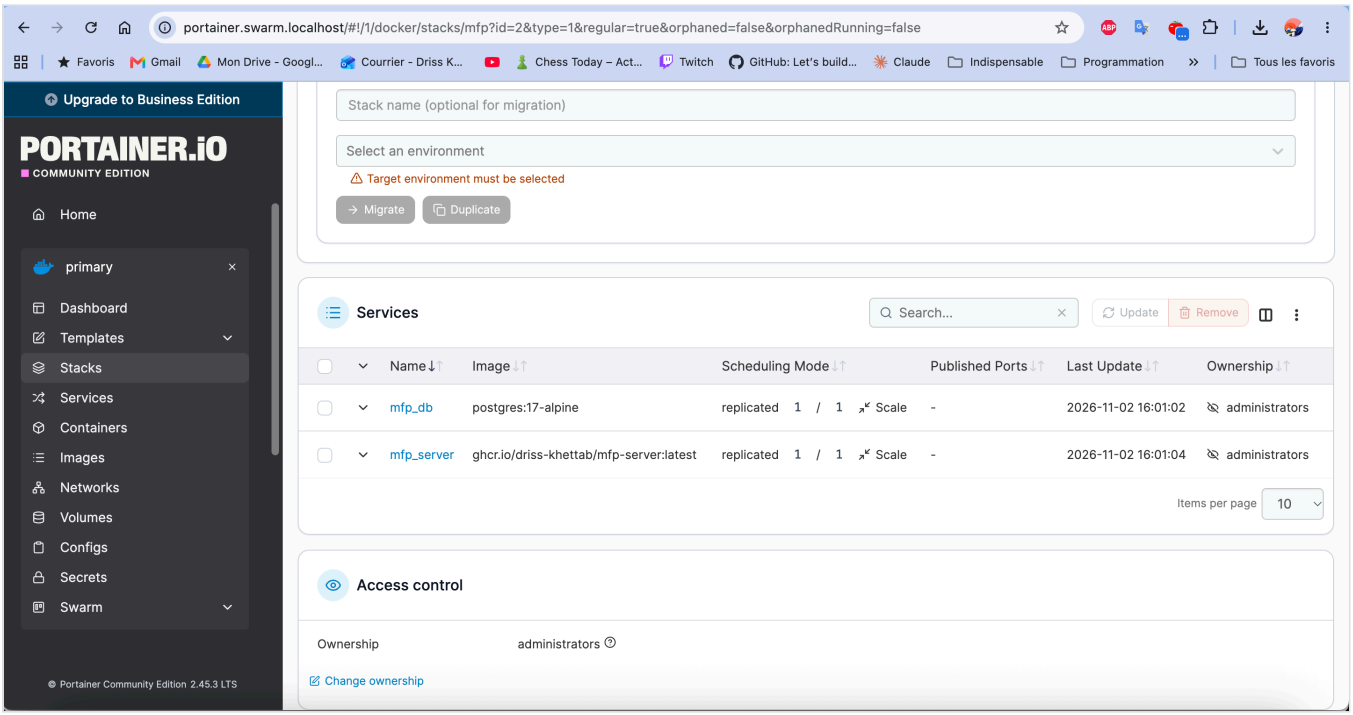


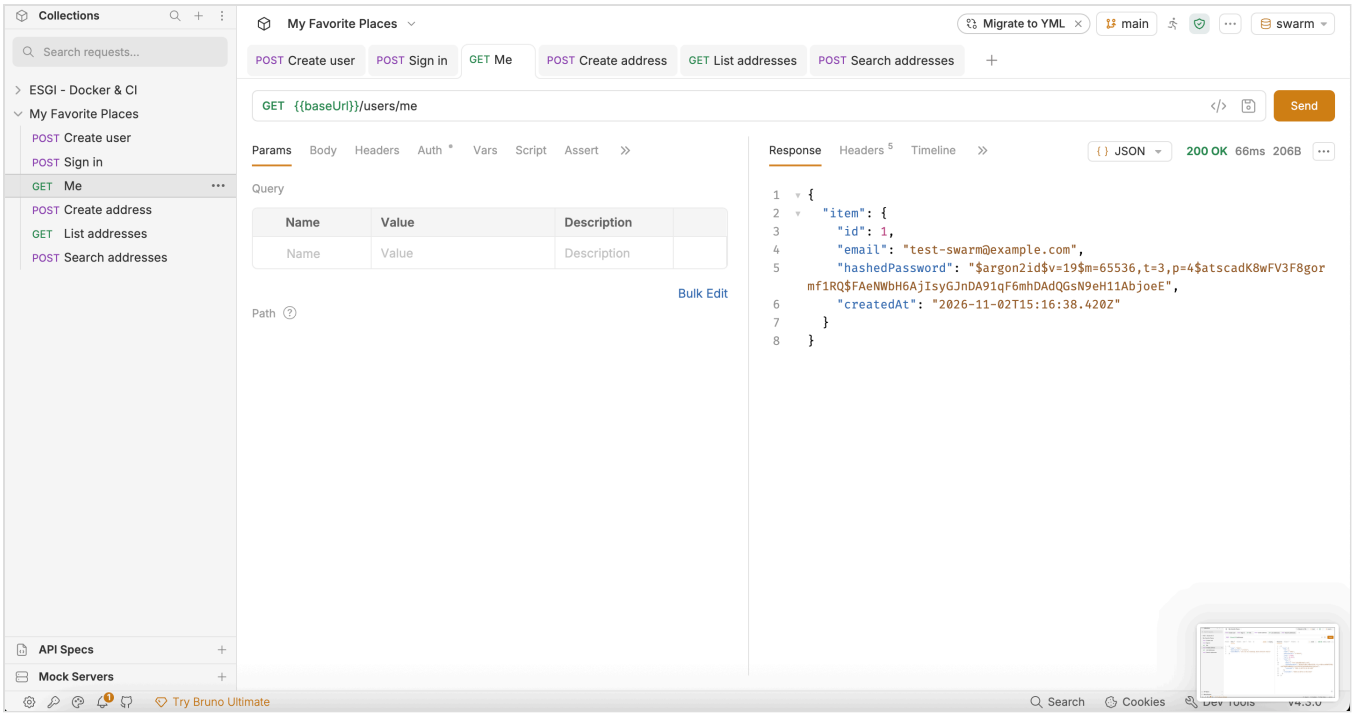
3. Vérifier dans le navigateur

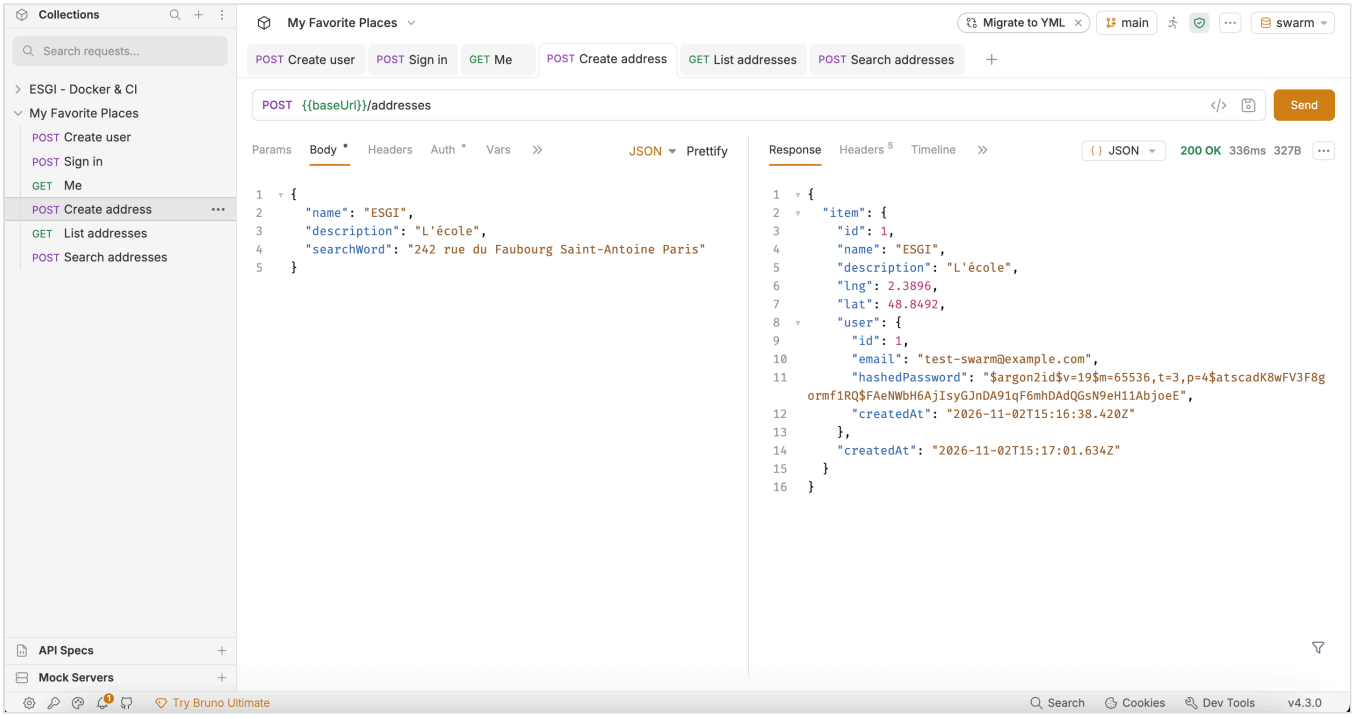




4. Nouvelle stack pour l'API MFP, testée avec Bruno







5 - Secrets et configs

Exercice 1 : déployer des logiciels

1. Dozzle avec un utilisateur en secret (`users.yml`), sur `dozzle.swarm.localhost`

```
(base) ➜  esgi-2603-my-favorite-places git:(main) ✗ grep swarm /etc/hosts
127.0.0.1 traefik.swarm.localhost whoami.swarm.localhost vote.swarm.localhost result.swarm.localhost
127.0.0.1 portainer.swarm.localhost mfp-api.swarm.localhost
127.0.0.1 dozzle.swarm.localhost authelia.swarm.localhost outline.swarm.localhost
```

Secrets

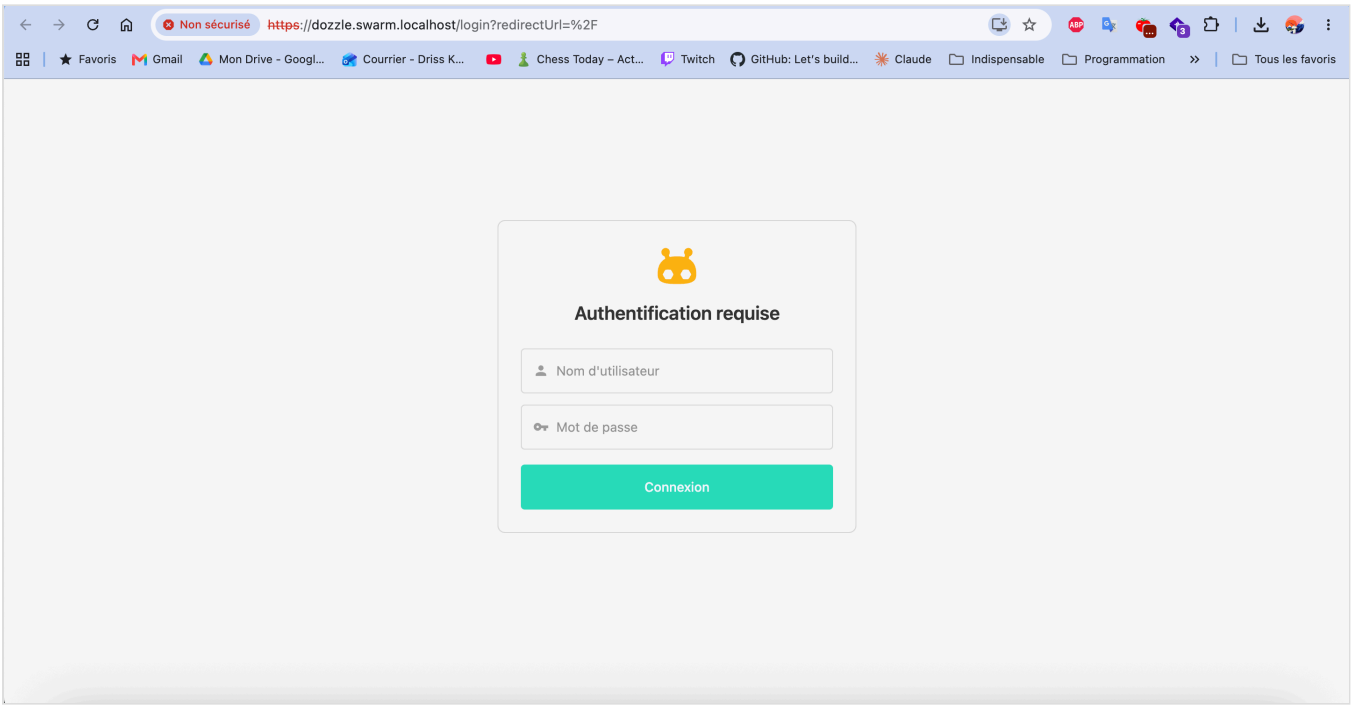
Search...

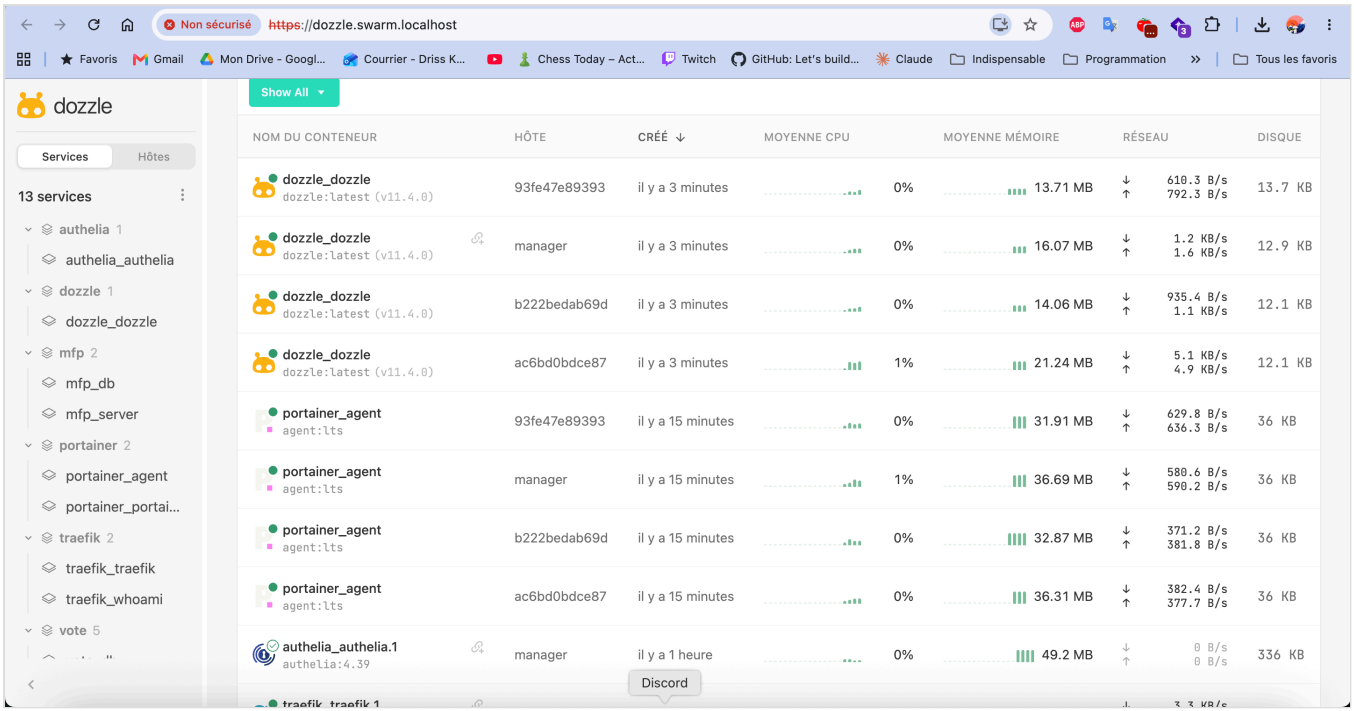
Remove

+ Add secret

Name	Creation Date	Ownership
swarm_localhost_key	2026-11-04 10:30:04	administrators
authelia_jwks	2026-11-04 10:32:17	administrators
authelia_storage_key	2026-11-04 10:32:17	administrators
dozzle_users	2026-11-04 10:30:27	administrators
authelia_oidc_hmac	2026-11-04 10:32:17	administrators
authelia_jwt_secret	2026-11-04 10:32:17	administrators
authelia_users	2026-11-04 10:32:17	administrators
authelia_session_secret	2026-11-04 10:32:17	administrators
dozzle_oidc_client_secret	2026-11-04 10:32:59	administrators

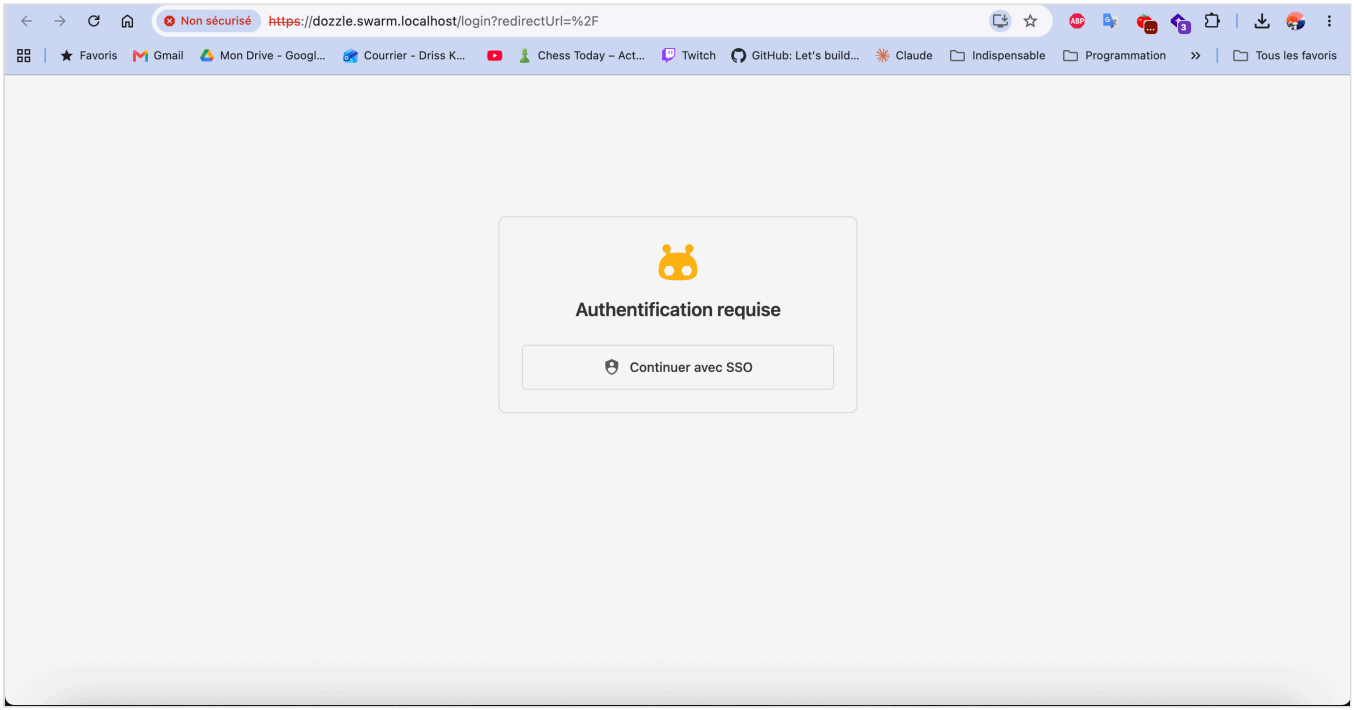
```
(base) ➜  esgi-2603-my-favorite-places git:(main) ✗ docker exec swarm-manager-1 docker stack services dozzle
ID             NAME             MODE         REPLICAS   IMAGE                          PORTS
vdtiwcd99mqi   dozzle_dozzle    global       4/4        ghcr.io/amir20/dozzle:latest
```

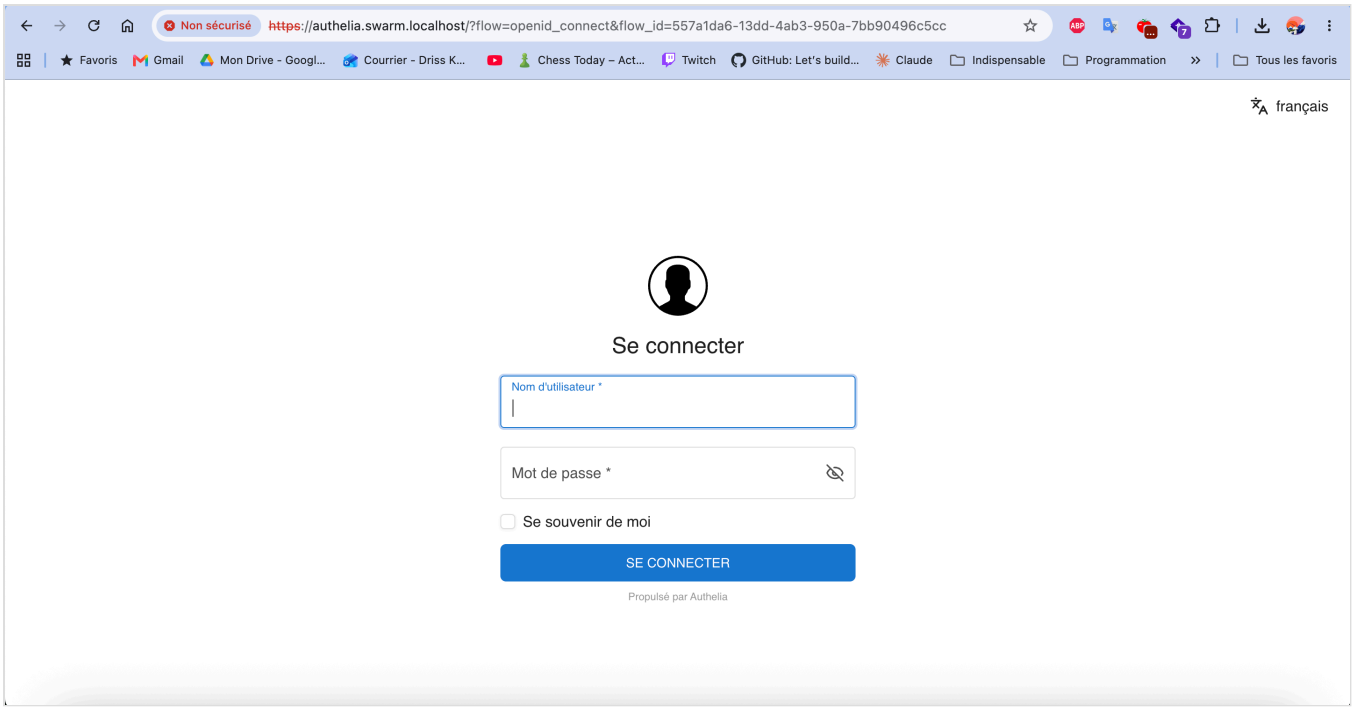


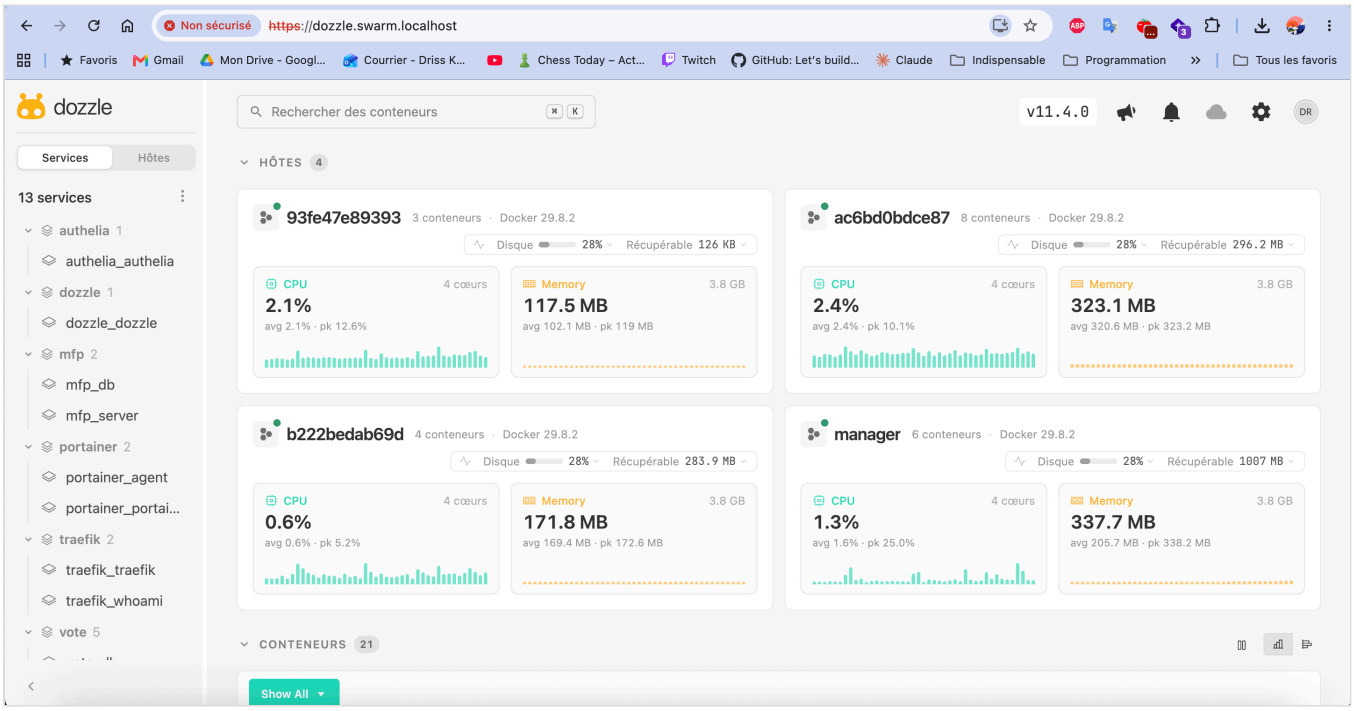


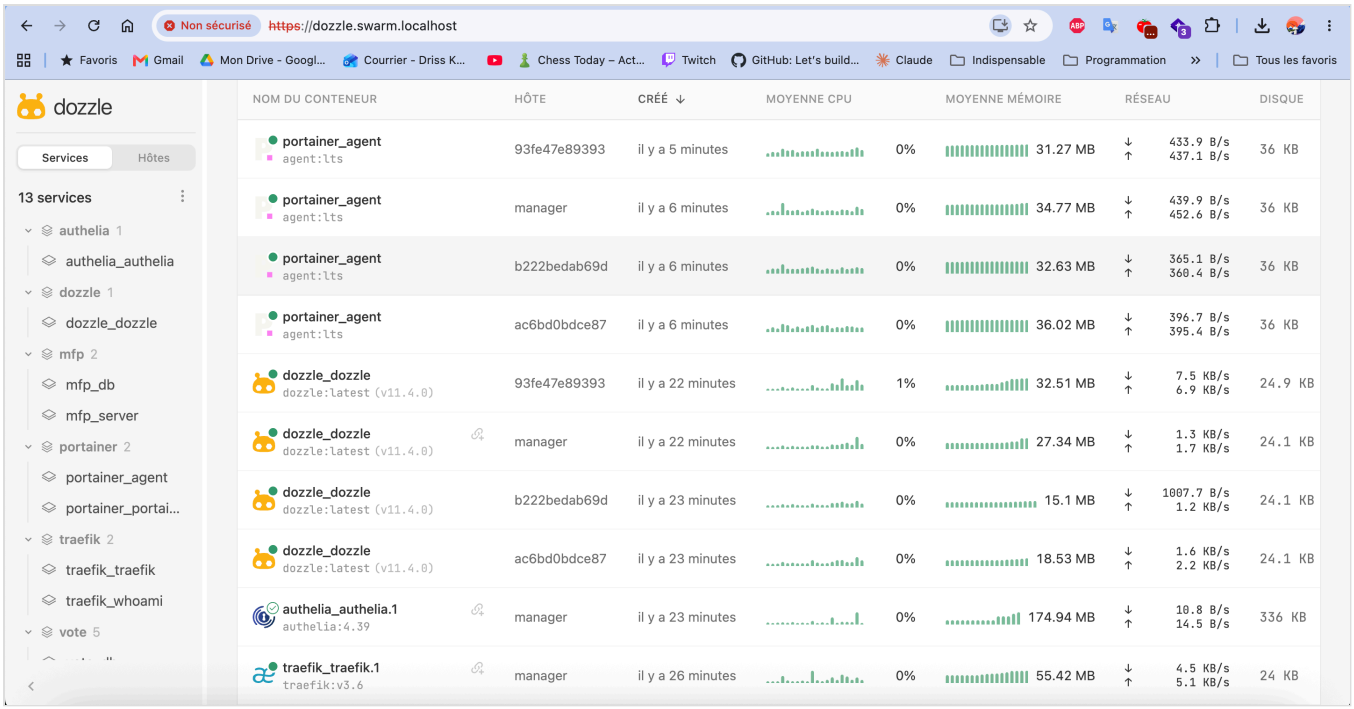
2. Authelia avec un compte en secret, et connexion à Dozzle en OIDC

```
(base) ➜  esgi-2603-my-favorite-places git:(main) ✗ docker exec swarm-manager-1 docker stack services authelia
ID             NAME                MODE         REPLICAS   IMAGE                          PORTS
58uzsroif5f3   authelia_authelia   replicated   1/1        ghcr.io/authelia/authelia:4.39
```




Non sécurisé
https://authelia.swarm.localhost/?flow=openid_connect&flow_id=557a1da6-13dd-4ab3-950a-7bb90496c5cc
Favoris
Gmail
Mon Drive - Googl...
Courrier - Driss K...
Chess Today – Act...
Twitch
GitHub: Let's build...
Claude
Indispensable
Programmation
Tous les favoris
français
Se connecter
Nom d'utilisateur *
Mot de passe *
Se souvenir de moi
SE CONNECTER
Propulsé par Authelia


Non sécurisé
https://dozzle.swarm.localhost
Favoris
Gmail
Mon Drive - Googl...
Courrier - Driss K...
Chess Today – Act...
Twitch
GitHub: Let's build...
Claude
Indispensable
Programmation
Tous les favoris
dozzle
Rechercher des conteneurs
v11.4.0
DR
Services
Hôtes
13 services
authelia 1
authelia_authelia
dozzle 1
dozzle_dozzle
mfp 2
mfp_db
mfp_server
portainer 2
portainer_agent
portainer_portai...
traefik 2
traefik_traefik
traefik_whoami
vote 5
HÔTES 4
93fe47e89393 3 conteneurs · Docker 29.8.2
Disque 28% Récupérable 126 KB
CPU 4 cœurs
2.1%
avg 2.1% · pk 12.6%
Memory 3.8 GB
117.5 MB
avg 102.1 MB · pk 119 MB
ac6bd0bdce87 8 conteneurs · Docker 29.8.2
Disque 28% Récupérable 296.2 MB
CPU 4 cœurs
2.4%
avg 2.4% · pk 10.1%
Memory 3.8 GB
323.1 MB
avg 320.6 MB · pk 323.2 MB
b222bedab69d 4 conteneurs · Docker 29.8.2
Disque 28% Récupérable 283.9 MB
CPU 4 cœurs
0.6%
avg 0.6% · pk 5.2%
Memory 3.8 GB
171.8 MB
avg 169.4 MB · pk 172.6 MB
manager 6 conteneurs · Docker 29.8.2
Disque 28% Récupérable 1007 MB
CPU 4 cœurs
1.3%
avg 1.6% · pk 25.0%
Memory 3.8 GB
337.7 MB
avg 205.7 MB · pk 338.2 MB
CONTENEURS 21
Show All



3. Bonus : Outline

Non réalisé.